

Architecture & Technical Design: capestone-app (CAP-001/002/003)

Architecture & Technical Design: capestone-app (CAP-001/002/003)

Document Header & Scope

Target repository: KallakrindaVikram/capestone-app

In-scope user stories (Jira):

- [EPMCDMETST-68076](#) — Add keyword search and pagination to course catalog (CAP-001)
- [EPMCDMETST-68084](#) — Enable course archiving (soft delete) for owners (CAP-002)
- [EPMCDMETST-68093](#) — Standardize API error responses and return correct HTTP status codes (CAP-003)

High-Level Design (HLD)

System Overview

Capestone App is a two-tier web application composed of a React SPA (client) and a Node/Express REST API (api) backed by SQLite via Sequelize. The client uses a Data abstraction (client/src/Data.js) to call REST endpoints and React Router v6 for navigation. Authentication uses HTTP Basic Auth; Express middleware validates credentials, loads the User, and assigns req.currentUser. The requested enhancements add catalog search/sort/pagination, implement soft-delete via an archived flag with owner-only archive/unarchive endpoints, and standardize error response shapes and status codes (notably 404 for missing/hidden courses).

Primary User Flows

- **Catalog browse with search/sort/pagination (CAP-001):** User opens catalog, optionally types a keyword and selects a sort; the UI debounces input, updates URL query params, calls GET /api/courses with page/limit/q/sort, and renders results plus next/prev controls and an empty state.
- **Owner archives/unarchives a course (CAP-002):** Course owner visits a course detail page; if owner, sees Archive/Unarchive button based on archived state; clicking performs PUT /api/courses/:id/archive or /unarchive and refreshes detail/catalog state.
- **Not found and standardized errors (CAP-003):** When course ID is invalid or inaccessible, API returns 404 with {error:"Course Not Found"}; the client routes to a Not Found page. Validation uses {errors:[...]} and auth/authorization use {error:"..."} consistently.

Architecture Diagram

```
1 flowchart LR
2   U[User Browser] -->|HTTPS| R[React SPA]
3   R -->|Fetch JSON| A[Express API (/api)]
4   A -->|Sequelize ORM| S[(SQLite DB)]
```

```

5
6 subgraph Client
7   R --> D[Data.js API wrapper]
8   R --> RR[React Router]
9 end
10
11 subgraph API
12   A --> AU[auth-user middleware]
13   A --> CR[courses routes]
14   A --> UR[users routes]
15 end
16
17 CR --> S
18 UR --> S
19

```

Sequence Diagram

```

1 sequenceDiagram
2   autonumber
3   participant B as Browser
4   participant UI as React UI
5   participant API as Express API
6   participant DB as SQLite (Sequelize)
7
8   Note over UI: CAP-001 Catalog load/search/sort/page
9   B->>UI: Navigate / (Catalog)
10  UI->>API: GET /api/courses?page=1&limit=20
11  API->>DB: Course.findAll(limit, offset, include User, where archived=false)
12  DB-->>API: rows
13  API-->>UI: 200 [Course...]
14  UI-->>B: Render cards + pager
15
16  B->>UI: Type keyword "react"
17  UI->>API: GET /api/courses?page=1&limit=20&q=react&sort=title_asc
18  API->>DB: Course.findAll(where title/desc LIKE, order title ASC, limit/offset)
19  DB-->>API: rows
20  API-->>UI: 200 [Course...]
21  UI-->>B: Render filtered list
22
23  Note over UI: CAP-002 Archive course
24  B->>UI: Click Archive
25  UI->>API: PUT /api/courses/12/archive (Basic Auth)
26  API->>DB: Course.findByPk(12)
27  DB-->>API: course
28  API-->>UI: 204 No Content
29  UI->>API: GET /api/courses/12 (Basic Auth)
30  API->>DB: Course.findByPk(12)
31  DB-->>API: course(archived=true)
32  API-->>UI: 200 course JSON
33
34  Note over UI: CAP-003 Not Found
35  B->>UI: Navigate /courses/999999
36  UI->>API: GET /api/courses/999999
37  API->>DB: Course.findByPk(999999)
38  DB-->>API: null
39  API-->>UI: 404 {"error":"Course Not Found"}
40  UI-->>B: Route to NotFound view
41

```

Low-Level Design (LLD)

Database Design

Existing Models

- **User** (api/models/user.js): id, firstName, lastName, emailAddress, password (hashed);
User.hasMany(Course)

- **Course** (api/models/course.js): id, title, description, estimatedTime, materialsNeeded, foreign key stored as column `userid` ; Course.belongsTo(User)

Schema Changes (CAP-002)

- Add boolean column `Courses.archived` NOT NULL DEFAULT 0 (false).
- Add indexes: `Courses.title` , `Courses.userid` ; optionally `Courses.archived` or composite indexes to optimize archived filtering.

Seed Update Strategy

- Update `api/seed/data.json` and `api/seed/database.js` to include varied titles/descriptions for search testing (e.g., react, node, sql, testing).
- Mark a small subset of seeded courses with `archived=true` to validate visibility rules.

API Specifications

Conventions

- **Base path:** /api
- **Validation errors:** HTTP 400 with `{"errors":[...]}`
- **Auth/Authorization errors:** HTTP 401/403 with `{"error":"..."}`
- **Not Found:** HTTP 404 with `{"error":"Course Not Found"}` for missing courses (CAP-003)

GET /api/courses

Item	Details
Description	Return courses with optional pagination, keyword search and sorting; excludes archived by default; supports <code>includeArchived=true</code> .
Auth	Public (no auth)
Query Params	• <code>page</code> (int, default 1, positive) • <code>limit</code> (int, default 20, positive, max 100) • <code>q</code> (string, optional; case-insensitive contains match on title/description) • <code>sort</code> (string, allowed: title_asc, title_desc, newest, oldest) • <code>includeArchived</code> (string; if "true" include archived; else exclude)
Responses	• 200: JSON array of courses with included User (exclude

	password/createdAt/updatedAt) and course exclude createdAt/updatedAt • 400: {"errors":["page must be a positive integer","limit must be <= 100"]}
Sequelize shape	where merges archived filter + q filter; order mapped from sort; limit / offset from page/limit; include User with excludes.

GET /api/courses/:id

Item	Details
Description	Return a course by id; 404 if missing. If archived, 404 unless requester is authenticated owner.
Auth	Optional; required for archived-owner access. Invalid credentials → 401 {error:"..."}
Responses	• 200: course JSON including archived flag and User • 404: {"error":"Course Not Found"} (missing OR archived & not owner)

PUT /api/courses/:id/archive

Item	Details
Description	Archive a course (soft delete). Owner-only.
Auth	Basic Auth required; must be owner
Responses	• 204: No Content • 403: {"error":"You are not authorised to archive this course."} • 404: {"error":"Course Not Found"}

PUT /api/courses/:id/unarchive

Item	Details
Description	Unarchive a course. Owner-only.
Auth	Basic Auth required; must be owner
Responses	• 204: No Content • 403: {"error": "You are not authorised to unarchive this course."} • 404: {"error": "Course Not Found"}

Frontend Architecture

Component Structure

- **Catalog page:** Courses (container) → CatalogToolbar (SearchInput + SortSelect) → CourseCardList → PaginationControls → EmptyState
- **Course detail:** CourseDetail → OwnerActions (Update/Delete + Archive/Unarchive) → ArchivedBadge

URL Query String State Management (CAP-001)

- Persist `q`, `sort`, `page`, `limit` using React Router v6 `useSearchParams`.
- On `q/sort` change, reset `page` to 1 and update URL; fetch courses on param changes.

Debounce Strategy

- Debounce keyword input only (default 350ms). Sort/pagination triggers immediate fetch.
- Commit debounced `q` to URL params and reset `page=1`.

Wireframes

Catalog View

```

1 CATALOG (/)
2 +-----+
3 | Header: [Logo]                                     [Sign In/Sign Out] |
4 +-----+
5 | Search: [_____]   Sort: [title_asc v]   Limit: [20 v] |
6 | |
7 | Results:
8 | +-----+ +-----+ +-----+
9 | | Course | | Course | | Course |
10 | | React Basics | | Node APIs | | SQL for Beginners |
11 | +-----+ +-----+ +-----+
12 |
13 | [Prev] Page 1 [Next]
14 |
15 | Empty state (when no matches):
16 |   "No courses match your search. Try a different keyword."
17 |
18 | [+ New Course] (existing tile/button)
19 +-----+

```

Not Found View

```

1 NOT FOUND (/notfound)
2 +-----+
3 | Header |
4 +-----+
5 | 404 - Not Found |
6 | The page you are looking for doesn't exist. |
7 | |
8 | [Go to Course Catalog] |
9 +-----+
10

```

Traceability Matrix

Story	Acceptance Criteria	Design Elements (Section References)
CAP-001 EPMCDMETST-68076	Query params page/limit/q/sort; keyword filter; 200 paged results; 400 errors array; UI search/sort reset page; pager + empty state	• LLD → API Specifications → GET /api/courses • LLD → Frontend Architecture → URL Query String State Management • LLD → Frontend Architecture → Debounce Strategy • Wireframes → Catalog View
CAP-002 EPMCDMETST-68084	archived default false; list excludes archived by default; archived detail hidden from non-owner; archive/unarchive 204; 403 error for non-owner; UI buttons + badge; hide archived in main list	• LLD → Database Design → Schema Changes • LLD → API Specifications → GET /api/courses, GET /api/courses/:id • LLD → API Specifications → PUT /api/courses/:id/archive, /unarchive • LLD → Frontend Architecture → Component Structure
CAP-003 EPMCDMETST-68093	GET /courses/:id returns 404 {error:"Course Not Found"}; validation 400 errors[]; auth	• LLD → API Specifications → Conventions + GET /api/courses/:id

401 error; authorization 403
error; frontend routes to
NotFound

- HLD → Sequence Diagram
(Not Found flow)
- Wireframes → Not Found
View